MODERN APPLICATION DEVELOPMENT - II
PROJECT REPORT

1. Student Details:

Name: Chittari Sri Siva Siddhartha Varma
Roll Number: 24f1002705
Email: 24f1002705@ds.study.iitm.ac.in/siddhartha12122002@gmail.com
About me: I am from Hyderabad, I am an Aerospace Engineer and I have just completed B.tech from Amity Noida, and I will be pursuing a PhD from IIT Kanpur from 2026 July. I aspire to build the most cutting edge intelligent defence equipment of tomorrow.

2. Project Details

Project Title: Placement Portal V2

Problem Statement:
You are required to build a Placement Portal Application (PPA) web application that allows Admin (Institute), Companies, and Students to interact with the system based on their roles.

Approach:
The app was built using two main components for its core functionalities: JS Vue for a front end app, and Python Flask for the back end. They interact via API calls. It allows each of the different kinds of users to perform actions and view data according to their roles.

3. AI/LLM Declaration

In a chronological order, The internal logic and file organization was initially defined by me and I used ChatGPT (GPT5) to refine the file structure and internal code organization. Beyond that, most of the code was written manually with some cumbersome parts being automated away, like the repetitive front end elements and debugging. Then I further used AI to generate synthetic data to put into the database to begin implementing, verifying and debugging features.

The extent of AI/LLM generated code would be 15-25%, which was mostly inline suggestions built into VSCode with Copilot, and some automated parts.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework, interacts with DB, receives requests, for CRUD operations
sqlite	DB language
Vue	Front end app, dynamically generates web pages and interfaces with flask to call API, gather information and display
Bootstrap 5	Frontend styling and responsive design
Flask-cache	Caching dashboards of student and company as they are the most code heavy queries
Flask-Login	User authentication and session management

5. Database Schema/ER Diagram

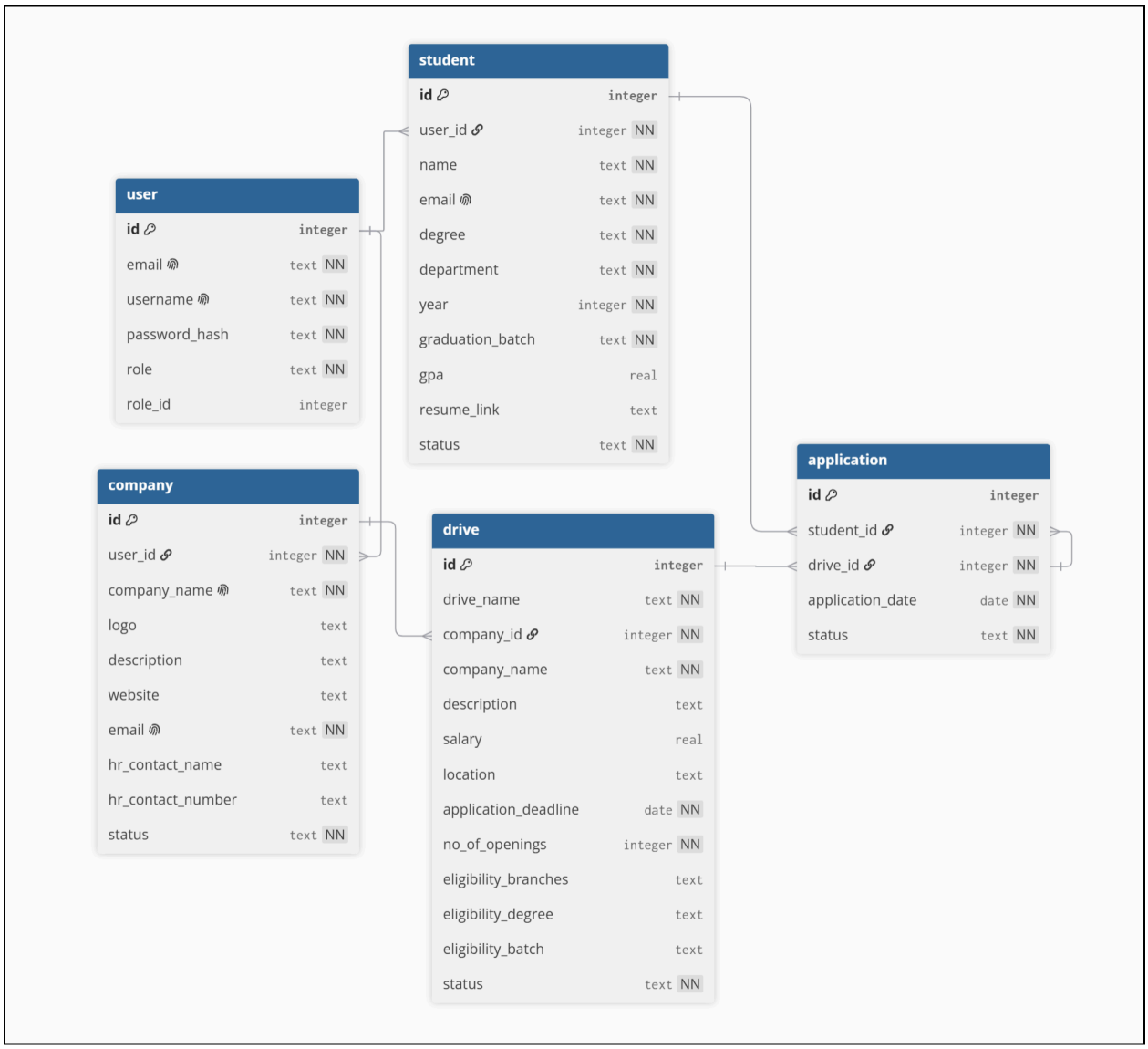
Tables:

- **User:** stores authentication and account details (id, email, username, password_hash, role, role_id)
- **Student:** stores student profile information (id, user_id, name, email, degree, department, year, graduation_batch, gpa, resume_link, status)
- **Company:** stores company profile information (id, user_id, company_name, logo, description, website, email, hr_contact_name, hr_contact_number, status)
- **Drive:** stores placement drive details created by companies (id, drive_name, company_id, company_name, description, salary, location, application_deadline, no_of_openings, eligibility_branches, eligibility_degree, eligibility_batch, status)
- **Application:** stores student applications submitted for placement drives (id, student_id, drive_id, application_date, status

Relationships:

- **One-to-One** → User → Student
    - Each student account is associated with exactly one user account.
- **One-to-One** → User → Company
    - Each company account is associated with exactly one user account.
- **One-to-Many** → Company → Drive
    - A company can create multiple placement drives.

- **One-to-Many** → Student → Application
  - A student can submit multiple applications.
- **One-to-Many** → Drive → Application
  - A placement drive can receive multiple applications.
- **Many-to-Many** → Student ↔ Drive (through Application)
  - Students can apply to multiple drives and each drive can have multiple student applicants.



6. API Resource Endpoints

Included in api.yaml file

7. Architecture and Features

Overview:

- app.py - main flask application entry point
- config.py - containing configuration information
- requirements.txt - contains requirements for functioning
- /instance - containing case specific information
- /application - main part containing all logic and orchestration
- /application/models - contain SQLAlchemy definition of each entity
- /application/routes - contains routes specific to each entity and function (auth, admin, student, company) purely an verifier and orchestrator. Routes call services and utils.
- /application/services - specific to each entity again, and contain the business logic, interacts with db_services to build queries
- /application/utils - only to process information, do not interact with external elements
- /application/services/db_services - functionally builds the query to send to db_executeror
- /application/executors - contains db_executor.py, this is the only function that interacts with the DB, built checks and fallbacks
- /application/tasks - contains all the celery tasks
- /application/template - contains index.html - entry point for Vue
- /application/static/js/app.js and router.js - main frontend app and router file for navigation
- /application/static/js/components - contains folder for each entity/section with necessary pages
- /application/celery_worker.py - the file defining celery beat functions and imports of tasks

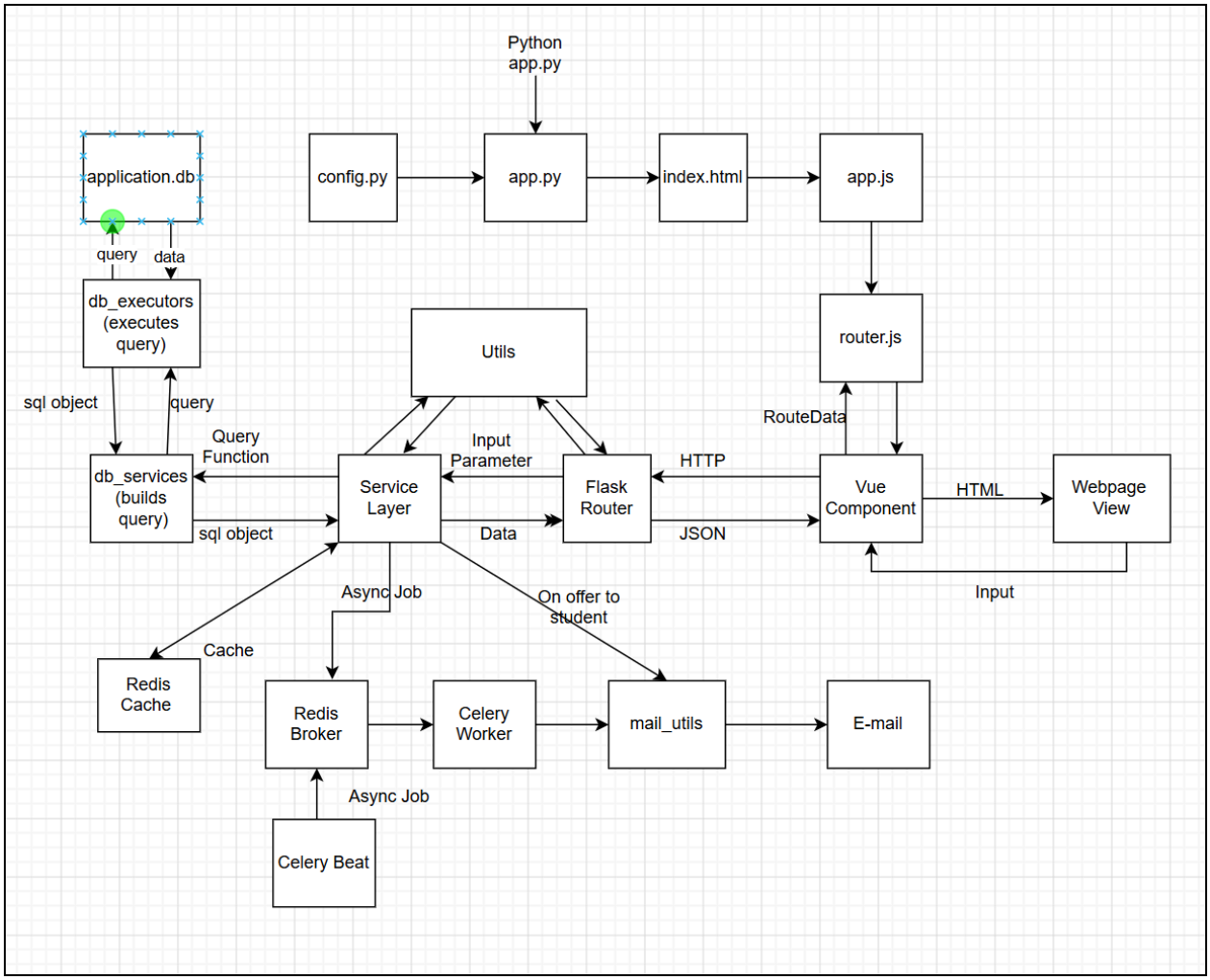
Implemented Features

- User registration and login for Students and Companies
- Session-based authentication and authorization
- Role-based access control (Admin, Student, Company)
- Student profile management
- Company profile management
- Placement drive creation and management
- Drive eligibility filtering based on degree, department, and graduation batch
- Student application submission and tracking

- Company application review and candidate shortlisting
- Offer acceptance and rejection workflow
- Student and company dashboards
- Administrative approval and moderation system
- Search functionality for students, companies
- Responsive frontend built using Vue.js and Bootstrap
- Secure session management

Additional Features

- **CSV Export System**
  - Students can export their application history.
  - Export generation is performed asynchronously using Celery.
- **Email Notification System**
  - Placement application exports delivered via email.
  - Offer notifications sent to students.
  - Automated reminder emails for upcoming deadlines.
  - Scheduled monthly placement reports sent to administrators.
- **Background Task Processing**
  - Celery worker integration for long-running operations.
  - Redis-backed task queue.
- **Scheduled Jobs**
  - Automated deadline reminder service.
  - Automated monthly placement analytics reports using Celery Beat.
- **Caching Layer**
  - Redis-based caching for dashboard and frequently accessed data.
  - Cache invalidation on data updates.
- **Database Design**
  - Normalized relational schema with User, Student, Company, Drive, and Application entities.
  - Foreign-key relationships with cascading deletes.
- **REST API Architecture**
  - Modular Blueprint-based API design.
  - JSON-based communication between frontend and backend.



8. Video Link

MAD-2 Project Video Report 24f1002705.mkv

https://drive.google.com/file/d/1_a5g6AepuN7u2kjcrNJ5j2mDK9aVDwDZ/view?usp=sharing